Our Approach to Implement the Game

1. Multithreading:
   We are using two threads for this game: the GameState thread and the main game thread.
   The GameState thread keeps track of which state the game is currently in — we have the main menu, running, and pause states.
   The thread for the main game logic handles the core game logic once the app enters the running state.

2. User Interface:
   We are using JavaFX to create the GUI of the game. The GameView class is responsible for animations and rendering. It updates the visuals and the positions of different entities, such as the moving enemy and collectibles in the game.
   We created different interactive buttons to enter the game from the main menu. The UI also displays the score, remaining collectibles, and other game information.

3. Map and Entity:
   The game world is represented by a 2D matrix — for example, # represents walls and E represents enemy spawn points.
   We use enums to define different cell types, such as FLOOR, WALL, or DOOR, each with its own collision priority. When multiple collisions happen at the same time, the one with the highest priority takes effect.

4. Collision Detection and Boundaries:
   The logic for detecting collisions works by checking if two items share the same 2D location (posX, posY).
   The boundaries of the game, which are the walls, have fixed locations. Once the checker detects that the player has the same location as a wall, it prevents the player from changing their 2D position, and the animation will not update.
   This logic is also used for the moving guardians (the enemies) to ensure that they do not cross the boundaries of the game.

5. Guardian Behavior:
   The implementation of the guardian has two parts: patrol mode and chasing mode.
   The patrol mode allows the guardian to move around the map, updating its 2D location every tick of the game.

Once the player enters the guardian's vision, it uses the shortest path algorithm in the binary matrix to chase the player's location.
When the guardian collides with the player, the game switches to the Game Over page, and the GameState changes to "Game Over," which stops the main game loop.

6. Collectibles:
   When the player's position overlaps with a collectible, position overlap detection triggers the pickup.
   The item becomes inactive after collection, the score updates, and the UI counter refreshes.
   A victory condition is checked when all collectibles are gathered.
   There is also visual feedback — the item immediately disappears, and a score notification is displayed.

Adjustments from Our Initial Plan

We decided to place the main game loop inside our GameView class so that we can update the animation at the same time as the actual entity movements.
We also separated the loop into two timelines instead of one for better management — one for the GameState and one for the main game.

Management Process including milestones

We divided our team into four divisions:

1. Core Game Engine and Entities:
   Responsible for collision detection, the cell system, and the entity classes.
   This member also implemented the moving guardian AI, including its vision and pathfinding.

2. UI/UX and JavaFX:
   Responsible for the ui package, which includes all screens, user input handling, and visual mockups.
   This member also worked on the game board and BoardView rendering.

3. Game System and Infrastructure:
   Responsible for the ScoreManager, Timer, and VisualManager.

4. Orchestration and Build Quality:
   Responsible for the overall project structure, including the pom.xml file, state management, and game loop.
   This member also handled documentation and testing for the entire game.

We set milestones for which parts needed to be completed before the midway deadline and the final deadline.
Before the midway deadline, we focused on implementing different functions of the game, such as player movement, system integration, UI mockups, the HUD, and the main game loop.
After the midway deadline, we focused on merging all components developed by different team members to ensure the game worked as expected and on resolving any remaining issues.

External libraries used:

JavaFX

The reason we use JavaFX is that its animation features bring dynamics and a modern look to the application interface.
It also allows us to use FXML to create the UI, which helps separate the user interface design from the application logic.
Additionally, the Scene Builder tool makes it much easier to visually design and organize the layout of the application without manually coding every element.

Jackson

The reason using Jackson to handle JSON data in our game since it provides an easy way to convert java objects to JSON format, we can make a level loader to save and manage the game data.

Measures of enhancing quality of code

We used separate branches on GitHub to prevent code conflicts. Since we worked on different components, this approach made merging faster and allowed us to discover issues early, helping to prevent bugs. Version control also lets us revert to previous versions if errors occur.

We notified our teammates whenever we finished a part of the project so everyone stayed updated on progress and understood what was happening. We divided roles instead of working all together, allowing everyone to focus on their own tasks and improving maintainability. This approach also reduces unnecessary changes to other team members' code, as we can add new functions without modifying existing ones.

Biggest challenges of this phase

One of the main challenges was learning and using external libraries, such as JavaFX and Jackson, since this was our first group project involving a GUI. Understanding how to work with these libraries, especially for animations and scene management, took time and practice.

Another challenge was that the game is runtime-driven rather than event-driven, which made the design and implementation more complicated. We had to carefully manage timelines, entity updates, and the game loop to ensure smooth gameplay while keeping the logic synchronized with the visuals.

Integration of components was also challenging because different team members developed different parts of the game independently. To prevent conflicts and bugs, we had to continuously coordinate, review each other's code, and test merged components frequently. This required ongoing communication and careful version control to maintain a stable and functional game.